

Tarefa 8: Falso Compartilhamento e Coerência de Cache

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Objetivo

Esta tarefa tem como objetivo implementar a estimativa de π pelo método de Monte Carlo em quatro variantes, combinando dois geradores de números aleatórios — `rand()` e `rand_r()` — com duas estratégias de acumulação de resultados — `#pragma omp critical` e vetor compartilhado. A análise foca nos fenômenos de contenção de mutex, falso compartilhamento (false sharing) e coerência de cache em sistemas multicore.

2. Ambiente de Execução

Item	Valor
CPU	AMD Ryzen 5 5600G
Núcleos físicos	4
Tamanho da cache line	64 bytes
Compilador	GCC com -O2 -fopenmp
Número de pontos (N)	100.000.000
Threads	4

3. Implementação

Cada versão combina um eixo de geração de números aleatórios com um eixo de acumulação do contador de acertos. O código completo segue abaixo para cada variante.

V1 — `rand()` + `#pragma omp critical`

Utiliza o gerador padrão `rand()`, cujo estado global no glibc é protegido por mutex. O `critical` é executado apenas 4 vezes (uma por thread), com custo desprezível.

```
long total = 0;
#pragma omp parallel num_threads(4)
{
    long local = 0;
    for (long i = 0; i < N/4; i++) {
        double x = rand() / (double)RAND_MAX;
        double y = rand() / (double)RAND_MAX;
        if (x*x + y*y <= 1.0) local++;
    }
    #pragma omp critical
```

```
total += local; /* executado apenas 4 vezes */
}
```

V2 — rand() + vetor compartilhado

Os acertos são acumulados em hits[NTHREADS]. Com 4 elementos long (4×8 = 32 bytes), o vetor cabe inteiramente em uma única cache line de 64 bytes, criando falso compartilhamento.

```
long hits[4] = {0}; /* 32 bytes → UMA cache line de 64 B */
#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();
    for (long i = 0; i < N/4; i++) {
        double x = rand() / (double)RAND_MAX;
        double y = rand() / (double)RAND_MAX;
        if (x*x + y*y <= 1.0) hits[tid]++; /* false sharing */
    }
}
```

V3 — rand_r() + #pragma omp critical

rand_r() recebe a seed por ponteiro e cada thread mantém sua própria seed na pilha — zero contenção no RNG. O critical continua sendo executado apenas 4 vezes.

```
#pragma omp parallel num_threads(4)
{
    unsigned seed = 42u ^ (unsigned)omp_get_thread_num();
    long local = 0;
    for (long i = 0; i < N/4; i++) {
        double x = rand_r(&seed) / (double)RAND_MAX;
        double y = rand_r(&seed) / (double)RAND_MAX;
        if (x*x + y*y <= 1.0) local++;
    }
    #pragma omp critical
    total += local;
}
```

V4 — rand_r() + vetor compartilhado

RNG sem contenção, mas o vetor hits[] ainda gera falso compartilhamento. A cada incremento de hits[tid], a cache line inteira é invalidada para os demais núcleos.

```
#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();
    unsigned seed = 42u ^ (unsigned)tid;
    for (long i = 0; i < N/4; i++) {
        double x = rand_r(&seed) / (double)RAND_MAX;
        double y = rand_r(&seed) / (double)RAND_MAX;
        if (x*x + y*y <= 1.0) hits[tid]++; /* false sharing */
    }
}
```

4. Resultados

Versão	RNG	Acumulação	Tempo (s)	π estimado
V1 — rand() + critical	rand()	critical	3,89	3,141724
V2 — rand() + vetor	rand()	vetor	4,84	3,141485
V3 — rand_r() + critical	rand_r()	critical	0,28	3,141599
V4 — rand_r() + vetor	rand_r()	vetor	0,59	3,141599

Referência: $\pi = 3,141593$

5. Análise

5.1 Coerência de cache: contenção no rand()

A função rand() no glibc mantém um estado global protegido por mutex. Com 4 threads gerando 2 números por ponto (total de 200 milhões de chamadas), cada invocação adquire e libera o mesmo lock. As threads passam a maior parte do tempo aguardando umas às outras para acessar o RNG — a paralelização torna-se ilusória.

Isso explica a diferença de aproximadamente 14× entre V1 (3,89 s) e V3 (0,28 s): ambas usam critical para acumulação, porém a V3 utiliza rand_r() com seed privada na pilha — zero contenção inter-thread.

5.2 Falso compartilhamento: vetor compacto

O array long hits[4] ocupa $4 \times 8 = 32$ bytes — metade de uma cache line de 64 bytes. Todos os 4 elementos residem na mesma linha de cache. Quando a thread 0 incrementa hits[0], o processador marca a linha inteira como modificada (protocolo MESI: estado Modified). Antes que a thread 1 possa escrever em hits[1] — em outro núcleo — ela precisa obter a linha atualizada, forçando uma transferência entre caches e invalidando a cópia local.

Esse fenômeno ocorre a cada incremento, gerando cerca de 25 milhões de invalidações por thread. O resultado é V4 (0,59 s) sendo mais lenta que V3 (0,28 s), apesar de V4 não possuir nenhum critical. O critical em V3 é executado apenas 4 vezes — custo irrisório frente a 100 milhões de invalidações de cache line em V4.

A V2 é a pior das quatro versões: acumula a contenção de mutex do rand() com o falso compartilhamento do vetor, resultando em 4,84 s.

5.3 Resumo dos gargalos

Versão	Gargalo principal	Tempo (s)
V1	Mutex do rand() — 200 M aquisições de lock	3,89
V2	Mutex do rand() + false sharing em hits[]	4,84
V3	Nenhum significativo (critical roda apenas 4 vezes)	0,28
V4	False sharing em hits[] (~100 M invalidações de cache)	0,59

5.4 Lição central

Eliminar o critical da acumulação (V3 → V4) piorou o desempenho em 2× porque o verdadeiro problema não era o critical — eram as escritas concorrentes em posições adjacentes de memória. Um critical executado 4 vezes custa infinitamente menos do que 100 milhões de invalidações de cache line.

Para corrigir o falso compartilhamento em V4 seria necessário preencher (pad) cada elemento do vetor até o tamanho da cache line, garantindo que cada hits[tid].val resida em sua própria linha:

```
typedef struct { long val; char pad[56]; } padded_t; /* 64 B por elemento */
padded_t hits[4];
```

6. Conclusão

Os experimentos demonstraram que o desempenho em programação paralela depende de fatores que vão além da simples remoção de seções críticas. O uso de rand() criou um gargalo de mutex tão severo que a versão com rand_r() e apenas 4 operações critical (V3) foi 14× mais rápida. O falso compartilhamento, por sua vez, transformou uma acumulação aparentemente livre de sincronização (V4) em algo 2× mais lento que a versão com critical (V3).

A versão V3 — rand_r() com seed privada por thread e acumulação via critical executado uma única vez por thread — apresentou o melhor resultado (0,28 s), demonstrando que minimizar o compartilhamento de estado e limitar as operações de sincronização ao estritamente necessário são os princípios centrais da programação paralela eficiente.

7. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <time.h>
6  #include <omp.h>
7
8  #define N      100000000L
9  #define NTHREADS 4
10 #define SEED    42u
11 #define PI_REF  3.14159265358979323846
12 #define BILLION 1000000000.0
13
14 static double elapsed(struct timespec a, struct timespec b) {
15     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
16 }
17
18 /* — V1: rand() + #pragma omp critical ————— */
19 static double v1_rand_critical(void) {
20     long total = 0;
21     #pragma omp parallel num_threads(NTHREADS)
22     {
23         long local = 0;
24         for (long i = 0; i < N / NTHREADS; i++) {
25             double x = rand() / (double)RAND_MAX;
26             double y = rand() / (double)RAND_MAX;
27             if (x*x + y*y <= 1.0) local++;
28         }
29         #pragma omp critical
30         total += local;
31     }
32     return 4.0 * total / N;
33 }
34
35 /* — V2: rand() + vetor compartilhado (falso compartilhamento) ————— */
36 static double v2_rand_vetor(void) {
37     long hits[NTHREADS];
38     for (int t = 0; t < NTHREADS; t++) hits[t] = 0;
39
40     #pragma omp parallel num_threads(NTHREADS)
41     {
42         int tid = omp_get_thread_num();
43         for (long i = 0; i < N / NTHREADS; i++) {
44             double x = rand() / (double)RAND_MAX;
45             double y = rand() / (double)RAND_MAX;
46             if (x*x + y*y <= 1.0) hits[tid]++; /* false sharing */
47         }
48     }
49
50     long total = 0;
51     for (int t = 0; t < NTHREADS; t++) total += hits[t];
52     return 4.0 * total / N;
53 }
54
```

```

1
2  /* — V3: rand_r() + #pragma omp critical ————— */
3  static double v3_randr_critical(void) {
4      long total = 0;
5      #pragma omp parallel num_threads(NTHREADS)
6      {
7          unsigned seed = SEED ^ (unsigned)omp_get_thread_num();
8          long local = 0;
9          for (long i = 0; i < N / NTHREADS; i++) {
10             double x = rand_r(&seed) / (double)RAND_MAX;
11             double y = rand_r(&seed) / (double)RAND_MAX;
12             if (x*x + y*y <= 1.0) local++;
13         }
14         #pragma omp critical
15         total += local;
16     }
17     return 4.0 * total / N;
18 }
19
20 /* — V4: rand_r() + vetor compartilhado (falso compartilhamento) ————— */
21 static double v4_randr_vetor(void) {
22     long hits[NTHREADS];
23     for (int t = 0; t < NTHREADS; t++) hits[t] = 0;
24
25     #pragma omp parallel num_threads(NTHREADS)
26     {
27         int tid = omp_get_thread_num();
28         unsigned seed = SEED ^ (unsigned)tid;
29         for (long i = 0; i < N / NTHREADS; i++) {
30             double x = rand_r(&seed) / (double)RAND_MAX;
31             double y = rand_r(&seed) / (double)RAND_MAX;
32             if (x*x + y*y <= 1.0) hits[tid]++; /* false sharing */
33         }
34     }
35
36     long total = 0;
37     for (int t = 0; t < NTHREADS; t++) total += hits[t];
38     return 4.0 * total / N;
39 }
40
41 int main(void) {
42     struct timespec ini, fim;
43
44     printf("Estimativa de  $\pi$  – Monte Carlo (N=%ldM, %d threads, cache line=%d B)\n\n",
45           N / 1000000L, NTHREADS, 64);
46     printf("%-28s %-6s %-11s %-10s %-8s\n",
47           "Versão", "RNG", "Acumulação", "Tempo(s)", " $\pi$ ");
48     printf("%-28s %-6s %-11s %-10s %-8s\n",
49           "-----", "-----", "-----", "-----", "-----");
50
51     #define MEDIR(label, rng, acum, expr) \
52         clock_gettime(CLOCK_MONOTONIC, &ini); \
53         double pi_#label = (expr); \
54         clock_gettime(CLOCK_MONOTONIC, &fim); \
55         printf("%-28s %-6s %-11s %-10.4f %.6f\n", \
56               #label, rng, acum, elapsed(ini, fim), pi_#label);
57
58     MEDIR(V1_rand_critical, "rand()", "critical", v1_rand_critical())
59     MEDIR(V2_rand_vetor, "rand()", "vetor", v2_rand_vetor())
60     MEDIR(V3_randr_critical, "rand_r()", "critical", v3_randr_critical())
61     MEDIR(V4_randr_vetor, "rand_r()", "vetor", v4_randr_vetor())
62
63     printf("\n $\pi$  referência: %.6f\n", PI_REF);
64     return 0;
65 }
66

```